§ 6. Эффективный перебор: встреча посередине

Экспоненциальный перебор иногда остаётся лучшей отправной точкой. При $n = 40$ полный просмотр подмножеств содержит около 10^{12} вариантов. Две половины по 20 элементов дают примерно по миллиону состояний. Выигрыш появляется, только если половины можно объединить без перебора всех пар: миллион на миллион вернул бы исходную проблему.

Метод meet-in-the-middle, или встреча посередине, сохраняет результаты одной части поиска и организует быстрый поиск совместимого дополнения. В этом параграфе мы отделим перечисление состояний от их объединения, разберём кратности, доминирование, XOR-пути и большой и малый шаг. Основной язык реализаций — C++17; из предыдущих параграфов нужны двоичный поиск, битовые маски и оценка памяти.

6.1. Контракт объединения двух половин

Разделим индексы на непересекающиеся множества L и R. Каждое подмножество исходных индексов единственным образом представляется парой подмножеств этих половин. Это даёт основу корректности: мы не теряем решения и не создаём лишних, если для пары правильно проверено исходное условие.

Для суммы условие $sum(X) + sum(Y) = T$ превращается в поиск $sum(Y) = T - sum(X)$. Для ограничения веса нужен поиск лучшей ценности при $weight(Y) \le W - weight(X)$. Для путей половины должны встретиться в одной вершине и согласовать значение общего состояния. Сам факт разбиения не говорит, какую сводку надо хранить.

Пусть числа состояний равны P и Q. Сортировка правой части и двоичный поиск для каждого левого состояния дают $O(Q \log Q + P \log Q)$ времени и $O(P + Q)$ памяти при хранении обоих списков. Если левую часть генерировать потоком, её список можно не хранить. Хеш-таблица часто даёт ожидаемое $O(P + Q)$, но гарантия зависит от модели хеширования; сортировка даёт детерминированную границу.

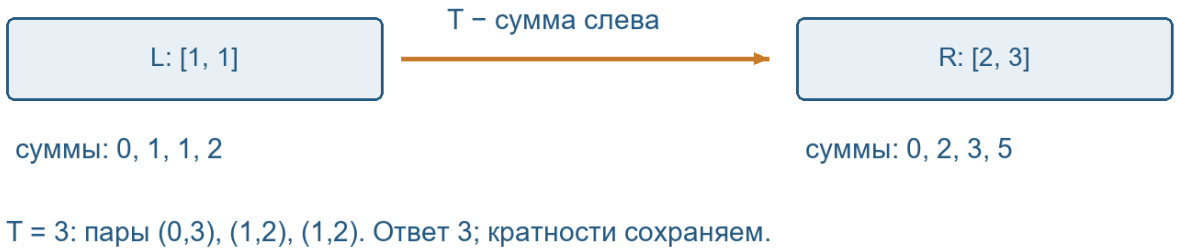


Рисунок 1: Состояние левой половины ищет совместимое дополнение в подготовленной правой половине.

При $|L| = \lfloor n/2 \rfloor$ и $|R| = \lceil n/2 \rceil$ размеры списков близки. Перекос делает большую половину дороже: при 10 и 30 элементах одна сторона содержит уже около миллиарда состояний. Если доступная память ограничивает сохранённую сторону, допускается несимметричное разбиение, но это осознанный обмен памяти на время.

(c) Aleksey Tolstikov

6.2. Пример курса: четыре числа с нулевой суммой

В старом конспекте дан массив $A = (2, 3, 1, 0, -4, -1)$ и разрешено использовать один элемент несколько раз. Например, $3 + 1 + 0 - 4 = 0$ и $0 + 0 + 0 + 0 = 0$. Эта оговорка принципиальна: она определяет допустимые пары и делает повтор индекса корректным.

Перепишем равенство как $a + b = -(c + d)$. Сохраним все n^2 сумм упорядоченных пар, отсортируем их, затем для каждой второй пары найдём противоположную сумму. Время $O(n^2 \log(n + 1))$, память $O(n^2)$. Чтобы восстановить ответ, вместе с суммой сохраним индексы пары. Для задачи существования можно остановиться при первом совпадении.

Если нужны четыре различных индекса, простая проверка наличия суммы уже недостаточна. Найденная пара может использовать тот же индекс, что и текущая. Нельзя объявить задачу решённой, проверив только одну произвольную пару на сумму: другая пара с той же суммой могла бы подойти. Такое ограничение требует дополнительной организации поиска или другого порядка обработки пар.

Для упорядоченных четвёрок кратности пар перемножаются. Подсчёт наборов индексов без учёта порядка — другая задача: нужно учитывать симметрии. Не делите ответ механически на $4!$: повторяющиеся индексы и значения создают разные размеры классов симметрии. Сначала зафиксируйте объект подсчёта.

Этот пример сохраняет постановку и числа письменного материала курса. Он удобен тем, что показывает метод без экспоненты: встреча посередине может уменьшать и степень полиномиального перебора.

6.3. Подсчёт подмножеств с заданной суммой

Считаем подмножества **индексов**, включая пустое, с суммой T. Равные значения на разных позициях дают разные выборы. Разрешены отрицательные числа: нельзя отбрасывать промежуточную сумму только потому, что она уже превышает T.

Для одной половины начинаем со списка `[0]`. Каждый новый элемент x создаёт копию прежних состояний с прибавленным x. Размер старой части запоминаем до добавлений: иначе новые состояния могли бы снова использовать тот же элемент. Суммарное число создаваемых записей равно $1 + 2 + \cdots + 2^k = O(2^k)$, а не $O(k2^k)$, как при отдельном просмотре всех битов каждой маски.

```
inline std::vector<long long> subset_sums(
        const std::vector<long long>& a,int l,int r) {
    assert(0<=l && l<=r && r<=int(a.size()) && r-l<=20);
    std::vector<long long> sums{0};
    for (int i=l; i<r; ++i) {
        auto old=sums.size();
        for (std::size_t j=0; j<old; ++j)
            sums.push_back(sums[j]+a[i]);
    }
    return sums;
}
```

Контракт полной функции: $n \le 40$, $|a_i| \le 10^9$, $|T| \le 10^{12}$. Эти ограничения гарантируют безопасность сумм и `target-x` в `long long`. Ответ не больше 2^{40} и также помещается в тип. Вспомогательный генератор предполагает не больше 20 элементов на половину.

```
inline long long count_subsets(const std::vector<long long>& a,
                               long long target) {
    assert(a.size()<=40);
    int n=int(a.size()), mid=n/2;
    auto left=subset_sums(a,0,mid);
    auto right=subset_sums(a,mid,n);
    std::sort(right.begin(),right.end());
    long long answer=0;
    for (long long x:left) {
        auto range=std::equal_range(right.begin(),right.end(),target-x);
        answer+=range.second-range.first;
    }
    return answer;
}
```

`equal_range` возвращает весь диапазон элементов с нужной суммой. Его длина — число правых подмножеств, дополняющих текущее левое. Замена отсортированного вектора множеством уникальных сумм сохранила бы ответы о существовании, но уничтожила бы кратности.

Разберём $[1, 1, 2, 3]$ и $T = 3$, разделив после второй позиции. Левые суммы: $[0, 1, 1, 2]$, правые: $[0, 2, 3, 5]$. Ноль слева сочетается с тройкой справа; каждая из двух единиц слева — с двойкой справа. Ответ 3. Выбор одного из двух элементов 1 различает два подмножества, хотя их суммы совпадают.

При пустом массиве генераторы возвращают по одному состоянию 0. Поэтому ответ равен 1 для $T = 0$ и 0 иначе. Это корректная обработка пустого подмножества, а не специальная ошибка подсчёта. Для сорока нулей и $T = 0$ ответ 2^{40} — хороший тест типа счётчика.

6.4. Рюкзак: почему недостаточно ближайшего веса

Теперь каждый предмет имеет неотрицательные вес и ценность; вместимость $W \geq 0$. Нужно максимизировать суммарную ценность при весе не больше W, каждый предмет используется не более одного раза. Обычное DP по вместимости неудобно, если веса порядка 10^9, но предметов не больше сорока.

Пример из видео 11.10.2022, 20:07–21:59. Лектор рассматривает размещение виртуальных машин на сервере: у заявок есть требования по оперативной памяти и CPU, а у сервера — отдельные пределы каждого ресурса. Приоритет заявки задаёт ценность. Это многомерный рюкзак, и его нельзя незаметно заменить одним ограничением на сумму весов. Наш код ниже решает одномерную учебную модель. Для двух ресурсов состояние имеет вид «память, CPU, ценность»; правило доминирования сравнивает оба расхода, а один двоичный поиск по весу уже не выбирает лучшее допустимое дополнение.

Состояние половины — пара (w, v). Для левого состояния нужно найти максимальное v_R среди правых с $w_R \leq W - w_L$. Простого двоичного поиска по весу недостаточно: наиболее тяжёлое допустимое состояние не обязано быть самым ценным.

Состояние (w_2, v_2) можно удалить, если есть (w_1, v_1) с $w_1 \leq w_2$ и $v_1 \geq v_2$: первая пара не тяжелее и не хуже по ценности. Для любого допустимого дополнения она остаётся допустимой и даёт не меньший ответ. Это доказательство доминирования относится к оптимизации значения; если требуется число оптимальных подмножеств, удаление состояния может потерять кратность.

Отсортируем пары по возрастанию веса, а при равном весе — по убыванию ценности. Оставим только строгие рекорды ценности. Тогда веса и ценности сохранённого фронта строго возрастают. Последнее состояние с допустимым весом одновременно имеет максимальную ценность среди допустимых.

Например, пары $(0,0),(2,4),(3,3),(3,5),(5,5),(6,9)$ после сортировки и фильтрации дают $(0,0),(2,4),(3,5),(6,9)$. Пара $(3,3)$ хуже $(2,4)$, а $(5,5)$ хуже $(3,5)$. При остатке вместимости 5 надо выбрать $(3,5)$, а не искать ровно вес 5.

6.5. Реализация рюкзака и её доказательство

Генератор может сразу не добавлять состояние тяжелее W, поскольку все дальнейшие веса неотрицательны. Для задачи с отрицательными весами это отсечение было бы неверным. Нулевая масса допустима; старый размер списка всё равно гарантирует однократность использования предмета.

```
struct Item { long long weight,value; };
using State=std::pair<long long,long long>; // weight, value
inline std::vector<State> enumerate_states(const std::vector<Item>& a,
                                           int l,int r,long long capacity) {
    std::vector<State> states{{0,0}};
    for (int i=l; i<r; ++i) {
        auto old=states.size();
        for (std::size_t j=0; j<old; ++j) {
            auto [w,v]=states[j];
            if (a[i].weight<=capacity-w)
                states.push_back({w+a[i].weight,v+a[i].value});
        }
    }
    return states;
}
```

В полной реализации $n \le 40$, веса и ценности лежат от 0 до 10^9, а $W \le 4\cdot 10^{10}$. Промежуточные суммы представимы в `long long`. Утверждения `assert` проверяют только часть предусловий и не заменяют ограничения входа.

```
inline long long best_knapsack(const std::vector<Item>& a,
                               long long capacity) {
    assert(a.size()<=40 && capacity>=0);
    for (auto item:a) assert(item.weight>=0 && item.value>=0);
    int mid=int(a.size())/2;
    auto right=enumerate_states(a,mid,int(a.size()),capacity);
    std::sort(right.begin(),right.end(),[](State x,State y) {
        return x.first!=y.first ? x.first<y.first : x.second>y.second;
    });
    std::vector<State> frontier;
    long long best=-1;
    for (auto state:right) if (state.second>best) {
        frontier.push_back(state); best=state.second;
    }
    auto left=enumerate_states(a,0,mid,capacity);
    long long answer=0;
    for (auto [w,v]:left) {
        long long room=capacity-w;
        auto it=std::upper_bound(frontier.begin(),frontier.end(),room,
            [](long long limit,State s){return limit<s.first;});
        assert(it!=frontier.begin());
        --it; answer=std::max(answer,v+it->second);
    }
    return answer;
}
```

Пустое подмножество гарантирует непустой список допустимых правых состояний. После фильтрации может остаться более ценное состояние нулевого веса; это тоже корректно. Поэтому при неотрицательном остатке вместимости `upper_bound` всегда имеет хотя бы одного предшественника.

Корректность. Любое допустимое решение распадается на перечисленные половины. Если его правая часть удалена, её доминирует сохранённая часть, которая совместима с той же левой и не уменьшает ценность. Для каждого левого состояния двоичный поиск находит лучший допустимый сохранённый вариант справа. Максимум по всем левым состояниям поэтому не меньше оптимума и сам является допустимым решением.

Время $O(n2^{\lceil n/2 \rceil})$ с учётом сортировки и поиска, память $O(2^{\lceil n/2 \rceil})$. Хранение масок для восстановления увеличивает константу памяти, но не порядок оценки. В этой реализации возвращается только значение оптимума; нельзя обещать восстановление состава предметов по одному числу.

6.6. XOR-пути и общий элемент двух половин

Пусть в прямоугольной таблице разрешены движения вправо и вниз. Нужно посчитать пути из левого верхнего угла в правый нижний, у которых XOR всех посещённых значений равен K. Каждый путь имеет $L = h + w - 2$ шагов. Выберем диагональ $i + j = \lfloor L/2 \rfloor$ и перечислим префиксы от старта и суффиксы от финиша до этой диагонали.

Для каждой клетки встречи храните частоты XOR префиксов. Смешивать состояния разных клеток нельзя: одинаковое XOR не означает, что части соединяются в один путь. Обратный перебор использует движения вверх и влево. Каждая полная траектория пересекает диагональ ровно один раз, поэтому разложение единственно.

Если обе половины включают значение клетки встречи a, то XOR целого пути равен $P \oplus R \oplus a$: в $P \oplus R$ общий элемент исчезает, и его надо вернуть один раз. Для данного R ищем частоту $P = K \oplus R \oplus a$. Можно иначе определить одну из половин без общей клетки, но соглашение должно быть единым в генерации и объединении.

Для таблицы с первой строкой $[1, 2, 3]$ и второй $[4, 5, 6]$ существуют три пути. Их XOR равны 6, 0 и 6. При $K = 6$ ответ 2. Через клетку со значением 2 префикс имеет $P = 1 \oplus 2 = 3$. Суффикс через верхний правый угол имеет $R = 2 \oplus 3 \oplus 6 = 7$; получаем $3 \oplus 7 \oplus 2 = 6$. Ошибочная формула без общей клетки дала бы 4.

Число частичных траекторий ограничено $O(2^{\lceil L/2 \rceil})$, хотя границы таблицы часто уменьшают его. При хешировании объединение имеет ожидаемую линейную стоимость по числу состояний; при сортировке частот добавляется логарифм. Счётчик путей должен вмещать число полных траекторий: заранее оцените биномиальный коэффициент или используйте заданный модуль. Это тематический разбор к задаче «Xor-пути», а не утверждение о прохождении серверных тестов конкретной реализации.

6.7. Углубление: большой и малый шаг

В мультипликативной группе по простому модулю p даны ненулевые g и h. Требуется найти $x \in [0, p-2]$, для которого $g^x = h \pmod{p}$, либо установить, что решения нет. Основание не обязано порождать всю группу; отсутствие решения возможно.

Положим $m = \lceil \sqrt{p-1} \rceil$ и запишем $x = im + j$, где $0 \le j < m$. Тогда $g^j = h(g^{-m})^i$. Сохраним малые шаги g^j, а затем последовательно вычислим правую часть для $i = 0, \dots, m-1$. Совпадение даёт кандидата $im + j$, который надо проверить на границу $p - 2$. При повторяющихся малых шагах для поиска наименьшего решения сохраняем минимальный j и выбираем минимальный допустимый кандидат.

Для $p = 23$, $g = 5$, $h = 8$ получаем $m = 5$. Малые шаги: $1, 5, 2, 10, 4$. Значение $5^5 = 20 \pmod{23}$ имеет обратный элемент 15. Первый большой шаг даёт $8 \cdot 15 = 5 \pmod{23}$, что совпадает с малым шагом $j = 1$. Получаем $x = 5 + 1 = 6$; действительно, $5^6 = 8 \pmod{23}$.

Таблица содержит $O(\sqrt{p})$ элементов. Сортировка и двоичный поиск дают $O(\sqrt{p} \log p)$ времени, хеширование — ожидаемое $O(\sqrt{p})$ после вычисления обратного множителя. Все умножения выполняются по модулю с защитой от переполнения. Для составного модуля и необратимого g эта формула без изменений не подходит: обратного элемента может не существовать.

Общий мотив тот же, что у сумм подмножеств: дорогой параметр разбивается на два меньших, одна сторона сохраняется, другая ищет совпадение. Однако доказательство использует свойства группы, а не разбиение множества предметов.

6.8. Восстановление решения и бюджет памяти

Для восстановления подмножества мало знать, что совместимая сумма существует. Сохраняйте вместе с суммой маску индексов своей половины. При расширении состояния новым элементом устанавливайте его локальный бит. Совпавшая пара масок однозначно восстанавливает исходные индексы: для правой половины к локальному номеру добавляется размер левой.

Для рюкзака маска должна перемещаться вместе с парой веса и ценности при сортировке. При удалении доминируемого состояния сохраняется хотя бы один не худший представитель. После двоичного поиска запоминаем обе маски, если найден лучший ответ. Если требуется определённый выбор среди равных оптимумов, например лексикографически минимальный список индексов, правило сравнения состояний надо дополнить; произвольный представитель может не выполнить такое требование.

Для подсчёта и для восстановления нужны разные объёмы информации. Все одинаковые суммы можно сжать в пару «сумма, частота», если нужен только счётчик. Для одного восстановленного ответа достаточно одной маски на сумму. Для перечисления всех решений могут понадобиться все маски или повторная генерация; сам вывод способен занимать экспоненциальное время.

Посчитаем память при половине из 20 элементов. Число состояний — $2^{20} = 1\,048\,576$. Список `long long` занимает 8 МиБ полезных данных. Две такие половины — 16 МиБ. Пара 64-битных веса и ценности обычно занимает 16 байт на состояние, то есть 16 МиБ на одну половину; структура с дополнительной маской может из-за выравнивания занимать 24 или 32 байта. Проверяйте `sizeof` конкретного типа, а не складывайте размеры полей на глаз.

Контейнер дополнительно хранит ёмкость. При геометрическом росте временно существуют старый и новый буферы; хеш-таблица имеет корзины и служебные данные. Фронт рюкзака может в худшем случае содержать все состояния: нельзя планировать память, надеясь, что доминирование всегда удалит почти всё. Потоковая генерация левой части экономит её список, но не уменьшает размер подготовленной правой стороны.

Отличайте ограничение числа предметов от величины весов. Для ста предметов meet-in-the-middle с двумя половинами по 50 непрактичен даже при маленьких весах. Тогда DP по сумме или битсет из § 4 может быть лучше. Для сорока предметов с огромными весами ситуация обратная: перебор половин зависит от числа предметов, а не от численного значения вместимости.

Контрольный расчёт. Сравните $n = 30$, $n = 40$ и $n = 50$. При увеличении n на десять каждая половина получает пять дополнительных элементов, поэтому число состояний увеличивается в 32 раза. Уменьшение константы вдвое не компенсирует такой рост. Именно поэтому перед реализацией оценивают не только асимптотику, но и конкретное число записей.

6.9. Двусторонний поиск и проверка границ метода

Поиск в графе тоже можно вести с двух концов. Если известна верхняя граница D длины искомого пути, полные BFS до глубин $\lfloor D/2 \rfloor$ от старта и $\lceil D/2 \rceil$ от финиша найдут общую вершину для любого пути длины не больше D. В ориентированном графе обратный поиск идёт по обратным рёбрам. Минимум суммы расстояний в пересечении даёт кратчайший путь в заданном ограничении.

При ограниченном ветвлении $b > 1$ это может сократить дерево поиска от порядка b^D до порядка $b^{\lceil D/2 \rceil}$. Оценка относится к числу достижимых состояний в модели; граф с большим пересечением путей и граф с огромной входящей степенью ведут себя иначе. Остановка при первом случайно найденном пересечении не заменяет доказательства для конкретной очередности расширения фронтов.

Для проверки сумм сравните результат с полным перебором при $n \leq 18$. Для рюкзака переберите все подмножества независимо и проверьте нулевые веса, одинаковые состояния, равные ценности при разных весах и предмет тяжелее вместимости. Для XOR-путей перечислите все пути на маленьких таблицах, включая таблицу 1×1.

Практика. Дополните рюкзак восстановлением индексов, сохраняя маски половин. Для XOR-путей явно выберите соглашение об общей клетке и напишите тест 1×1. Сравните память списка сумм, списка пар и списка пар с масками до запуска больших случаев. В § 7 мы будем сохранять не состояния перебора, а доступные версии одной структуры данных.

6.10. Практика к главе

В расширенной подборке приложения A: №3749 «Задача о рюкзаке» (основная); №4475 «Xor-пути» (основная); №4491 «Оптимальная подпоследовательность» (основная); №4474 «Лотерея» (углубление); №4524 «Разложение в сумму квадратов» (резерв КР); №4787 «Весы и гири» (резерв КР). Полные условия доступны по ссылкам в приложении. Выберите основные задачи, затем переходите к углублению; решать весь список за одну неделю не требуется.

6.11. Вопросы для самопроверки

1. Почему простое разбиение пополам не помогает, если затем перебираются все пары состояний?
2. Какое свойство обеспечивает единственность разложения подмножества?
3. Почему нельзя удалять одинаковые суммы при подсчёте?
4. Почему в задаче о среднем подмножества недостаточно хранить только его сумму? Как выразить допустимый диапазон сумм через число выбранных элементов?
5. При каких предпосылках можно отсекать слишком тяжёлые состояния?
6. Докажите безопасность удаления доминируемой пары для максимума ценности.
7. Почему для подсчёта оптимальных решений того же доказательства недостаточно?
8. Откуда в формуле XOR появляется значение клетки встречи?
9. Где в алгоритме большого и малого шага используется обратимость?
10. Почему обратный BFS в ориентированном графе идёт по обратным рёбрам?

(c) Aleksey Tolstikov